P.Vishwa nandan(192473015)

Q5 – Race Conditions in a Cinema Seat-Booking System

1. Aim

To implement a cinema seat-booking system using Java multithreading and understand race conditions. To compare unsynchronized booking, synchronized methods, and synchronized blocks to prevent double booking.

2. Problem Description

A cinema application allows multiple users to book seats simultaneously.

Each user checks whether a seat is available and then marks it as booked.

When multiple users try to book the same seat at the same time, a race condition may occur. This can result in the same seat being sold more than once.

To prevent this problem, Java synchronization techniques are used.

3. Concepts Used

Race Condition: A situation where multiple threads access shared data simultaneously and produce an incorrect result.

Synchronization: A mechanism that allows only one thread at a time to execute a critical section protected by the same lock.

Critical Section: The part of the program where shared data is accessed or modified.

Synchronized Method: A method that allows only one thread at a time to execute it using the object's intrinsic lock.

Synchronized Block: A block of code that allows only one thread at a time to execute it using a specified object's lock.

4. Java Program

File Name: SeatBooking.java

The following program compares three approaches:

1. Unsynchronized booking.
2. Synchronized method.
3. Synchronized block.

It also measures the execution time for 10,000 booking requests.

```
import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;
```

```
import java.util.concurrent.CountDownLatch;

class SeatBooking {

    private final boolean[] booked;
    private int bookingsMade = 0;

    SeatBooking(int seats) {
        booked = new boolean[seats];
    }

    // Version 1: Without synchronization
    public void bookWithoutSync(int seat) {

        if (!booked[seat]) {

            Thread.yield();

            booked[seat] = true;
            bookingsMade++;
        }
    }

    // Version 2: Synchronized method
    public synchronized void bookWithMethod(int seat) {

        if (!booked[seat]) {

            booked[seat] = true;
```

```
            bookingsMade++;
        }
    }

    // Version 3: Synchronized block
    public void bookWithBlock(int seat) {

        synchronized (this) {

            if (!booked[seat]) {

                booked[seat] = true;
                bookingsMade++;
            }
        }
    }

    public int getBookingsMade() {
        return bookingsMade;
    }
}

public class SeatBookingDemo {

    static void runTest(String type) throws InterruptedException {

        SeatBooking screen = new SeatBooking(10);

        int users = 100;
```

```
    Thread[] threads = new Thread[users];

    for (int i = 0; i < users; i++) {

        final int seat = i % 10;

        threads[i] = new Thread(() -> {

            if (type.equals("UNSYNC")) {
                screen.bookWithoutSync(seat);
            }
            else if (type.equals("METHOD")) {
                screen.bookWithMethod(seat);
            }
            else {
                screen.bookWithBlock(seat);
            }
        });

        threads[i].start();
    }

    for (Thread t : threads) {
        t.join();
    }

    System.out.println(type + " - Seats sold: " +
            screen.getBookingsMade());
}
```

```
static void measureTime(String type)
        throws InterruptedException {

    SeatBooking screen = new SeatBooking(1000);

    int requests = 10000;

    ExecutorService executor =
            Executors.newFixedThreadPool(100);

    CountDownLatch start = new CountDownLatch(1);

    long begin = System.nanoTime();

    for (int i = 0; i < requests; i++) {

        final int seat = i % 1000;

        executor.execute(() -> {

            try {
                start.await();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }

            if (type.equals("UNSYNC")) {
```

```
                screen.bookWithoutSync(seat);
            }
            else if (type.equals("METHOD")) {
                screen.bookWithMethod(seat);
            }
            else {
                screen.bookWithBlock(seat);
            }
        });
    }

    start.countDown();

    executor.shutdown();

    while (!executor.isTerminated()) {
        Thread.sleep(10);
    }

    long end = System.nanoTime();

    System.out.println(type + " - Time: " +
            (end - begin) / 1000000 + " ms");

    System.out.println(type + " - Seats sold: " +
            screen.getBookingsMade());
}

public static void main(String[] args)
```

```
        throws InterruptedException {

    System.out.println("DOUBLE BOOKING TEST");

    runTest("UNSYNC");
    runTest("METHOD");
    runTest("BLOCK");

    System.out.println("\n10,000 BOOKING REQUESTS");

    measureTime("UNSYNC");
    measureTime("METHOD");
    measureTime("BLOCK");
  }
}
```

5. Expected Output

The exact execution time and unsynchronized booking count may vary between executions.

One possible output is:

```
DOUBLE BOOKING TEST
UNSYNC - Seats sold: 18
METHOD - Seats sold: 10
BLOCK - Seats sold: 10

10,000 BOOKING REQUESTS
UNSYNC - Time: 250 ms
UNSYNC - Seats sold: 1000
METHOD - Time: 300 ms
METHOD - Seats sold: 1000
BLOCK - Time: 280 ms
```

BLOCK - Seats sold: 1000

Note: The execution times and unsynchronized result above are illustrative examples, not guaranteed results. Record the actual values from your computer.

The synchronized method and synchronized block must never count more than 10 successful bookings in the first test because only 10 seats are available.

6. Explanation of the Program

A. Without Synchronization

In this method, multiple threads can access the same seat simultaneously.

Two users may check the seat before either user marks it as booked.

Both users may then mark the same seat as booked and increase the booking counter.

This creates a race condition.

B. Synchronized Method

The synchronized keyword is added to the booking method.

public synchronized void bookWithMethod(int seat)

Only one thread at a time can execute this method on the same SeatBooking object.

Therefore, another user must wait until the current booking operation finishes.

This prevents double booking.

C. Synchronized Block

The synchronized block protects only the critical section.

```
synchronized (this) {


   if (!booked[seat]) {
      booked[seat] = true;
      bookingsMade++;
   }
}
```

Here, this represents the current SeatBooking object.

Only the check-and-book operation is protected by the lock.

D. Measuring Execution Time

The program uses System.nanoTime() to calculate the time taken to process 10,000 booking requests.

The fixed thread pool creates 100 worker threads to process the requests.

The CountDownLatch allows the submitted tasks to begin together.

7. Student A – Producing and Explaining the Race

1. How can double booking be reproduced?

Create 100 threads and allow them to compete for only 10 seats.

In the unsynchronized method, Thread.yield() gives other threads an opportunity to execute between checking and booking.

This increases the possibility of multiple threads accessing the same seat before it is marked as booked.

2. Recording Double Bookings Over Ten Runs

Run the unsynchronized version ten times and record the observed booking count.

Sample observation:

Run	Available Seats	Observed Bookings
1	10	16
2	10	18
3	10	14
4	10	17
5	10	15
6	10	19
7	10	13
8	10	16
9	10	18
10	10	15

Observation: The unsynchronized version may produce more than 10 bookings because multiple threads can pass the availability check at the same time.

The values above are illustrative. Replace them with your actual observations.

3. Race Condition Using Two Threads

Consider two users trying to book Seat 1.

1. Thread A checks Seat 1 and finds it available.
2. Thread B also checks Seat 1 and finds it available.
3. Thread A marks Seat 1 as booked.
4. Thread B also marks Seat 1 as booked.
5. Both threads increase the booking counter.

As a result, the same seat is counted as sold twice.

4. Identify the Critical Section

The following statements form the critical section:

```
if (!booked[seat]) {
   booked[seat] = true;
   bookingsMade++;
}
```

These statements must execute atomically to prevent double booking.

8. Student B – Comparing Synchronization Strategies

1. Measuring Time for 10,000 Bookings

The program measures the execution time of all three approaches using System.nanoTime().

The synchronized versions may take slightly more time because threads must acquire and release locks.

Actual performance depends on the processor, JVM, operating system, and thread contention.

2. Synchronized Method vs Synchronized Block

Synchronized Method:

- Locks the entire method.
- Uses the current object's intrinsic lock.
- Simple to implement.
- Other threads must wait until the method finishes.

Synchronized Block:

- Locks only the required critical section.

- Uses the specified object's intrinsic lock.
- Allows unrelated operations outside the block to execute without holding that lock.
- Provides more control over synchronization.

3. Static Synchronization

A static synchronized method locks the class object rather than an individual object.

Example:

```
public static synchronized void bookSeat() {
   System.out.println("Booking seat");
}
```

Static synchronization is useful when multiple objects of the same class share static data.

For example, if multiple SeatBooking objects represent screens that share one static seat counter, class-level synchronization may be required to protect that shared counter.

4. Intrinsic Lock

Every Java object has an intrinsic lock, also called a monitor lock.

- A synchronized instance method uses the current object's lock.
- A synchronized block using synchronized(this) uses the current object's lock.
- A static synchronized method uses the class object's lock.

Only one thread can hold the same intrinsic lock at a time.

9. Joint Engineering Decision

For a cinema with 20 screens, I would use a synchronized block to protect the seat availability check and booking operation.

It protects the critical section while allowing other operations outside that section to execute without holding the lock.

Each screen should have its own booking object so that users booking seats on different screens do not unnecessarily block each other.

For a real cinema application with multiple servers, database transactions and unique constraints should also be used because Java synchronization only protects threads within the same application process.

10. Alternative Concept – AtomicBoolean, ReentrantLock and ConcurrentHashMap

Other Java approaches can also prevent double booking.

A. AtomicBoolean

An AtomicBoolean provides the compareAndSet() method.

It checks whether a seat is available and changes its value atomically.

Example:

```
import java.util.concurrent.atomic.AtomicBoolean;

class AtomicSeatBooking {

   AtomicBoolean[] seats = new AtomicBoolean[10];

   AtomicSeatBooking() {

      for (int i = 0; i < seats.length; i++) {
         seats[i] = new AtomicBoolean(false);
      }
   }

   void book(int seat) {

      if (seats[seat].compareAndSet(false, true)) {
         System.out.println("Seat booked successfully");
      }
      else {
         System.out.println("Seat already booked");
      }
   }
}
```

B. ReentrantLock

ReentrantLock provides explicit lock and unlock operations.

Example:

```
import java.util.concurrent.locks.ReentrantLock;

class LockBooking {

    private final ReentrantLock lock = new ReentrantLock();

    void book() {

        lock.lock();

        try {
            System.out.println("Seat booked");
        }
        finally {
            lock.unlock();
        }
    }
}
```

The finally block ensures that the lock is released even if an exception occurs.

C. ConcurrentHashMap

ConcurrentHashMap provides putIfAbsent() to insert a seat only if it does not already exist.

Example:

```
import java.util.concurrent.ConcurrentHashMap;

class MapBooking {

    ConcurrentHashMap<Integer, String> seats =
            new ConcurrentHashMap<>();
```

```
    void book(int seat) {

        String result = seats.putIfAbsent(seat, "BOOKED");

        if (result == null) {
            System.out.println("Seat booked successfully");
        }
        else {
            System.out.println("Seat already booked");
        }
    }
}
```

11. Complexity Analysis

Let s represent the total number of seats.

Time Complexity

For a single booking attempt:

- Checking seat availability takes O(1).
- Updating the seat status takes O(1).
- Synchronization or lock acquisition may add waiting time under contention.

Therefore, the expected time per booking is **O(1)** when considering the basic seat lookup and update.

Under heavy contention, threads may spend additional time waiting for locks.

Space Complexity

The boolean array stores one booking status for every seat.

Therefore, the main storage requirement is:

O(s)

Additional synchronization objects or atomic variables also require memory proportional to the number of seats for the corresponding designs.

Approach	Time Per Booking (Expected)	Extra Space

Synchronized method	O(1)	O(s)
Synchronized block	O(1)	O(s)
AtomicBoolean array	O(1)	O(s)
ReentrantLock	O(1)	O(s)
ConcurrentHashMap	O(1) average	O(s)

12. Test Cases

Test Case	**Condition**	**Expected Result**
1	Two users book the same seat	Only one booking succeeds with synchronization
2	100 users compete for 10 seats	Maximum 10 successful bookings
3	Run without synchronization	Double booking may occur
4	Use synchronized method	Double booking prevented
5	Use synchronized block	Double booking prevented
6	Process 10,000 booking requests	Execution time is measured
7	Book seats on separate screen objects	Independent objects use separate locks

13. Conclusion

This experiment demonstrates race conditions in a cinema seat-booking system using Java multithreading.

The unsynchronized version may allow multiple users to book the same seat. Synchronized methods and synchronized blocks prevent this problem by protecting the critical section.

AtomicBoolean, ReentrantLock, and ConcurrentHashMap provide alternative approaches to safe concurrent booking.

Synchronization is essential for maintaining data consistency and preventing double booking in concurrent applications.